

# Veridian

## 工作空间与数据同步架构设计

### Workspace & Data Synchronization Architecture Design

---

控制面/数据面分离 • 可插拔同步后端 • 基于操作日志的合并 • 零强制云依赖

本方案在现有 Veridian 本地优先架构（§ 见 [readme/refactor/redesign.tex](#)）基础上，设计 Notion 式多人协作工作空间（邀请制、角色权限）与两种免费数据同步方式（GitHub 仓库 / 云盘文件夹），并逐条对照软件设计七大基本原则给出实现依据。

Contents

1 背景与目标 2

1.1 需求原文 2

1.2 v1 范围声明 2

2 总体架构：控制面与数据面分离 2

2.1 分层架构图 3

3 控制面设计：工作空间、成员、角色 3

3.1 为什么选 Supabase 3

3.2 控制面数据模型 3

3.3 角色权限矩阵 4

3.4 邀请流程时序 5

4 数据面设计：可插拔同步后端 5

4.1 SyncBackend 抽象 5

4.2 GitSyncBackend（GitHub 仓库） 6

4.3 CloudFolderSyncBackend（云盘文件夹） 6

4.4 冲突解决策略（v1：最后写入优先 + 手动提示） 6

5 权限与安全 7

5.1 PermissionSource 抽象 7

5.2 令牌与密钥的存储原则 7

6 客户端架构改动清单 7

7 七大设计原则逐一对应 8

8 与现有重构路线图的衔接 9

9 分阶段实施路线图 10

10 成本与限制 10

11 待扩展功能（Future Work） 11

# 1 背景与目标

## 1.1 需求原文

- 实现 Notion 式工作空间：可新建邀请制多人协作工作空间（邀请他人、分配权限），也可新建私人工作空间（个人多设备同步）。
- 数据同步方式一：在 GitHub 上创建仓库，用于团队共同管理和访问。
- 数据同步方式二：免费、无需承担费用、操作方便的备用同步方式。
- 权限模型：自建轻量控制面服务管理账号、成员、角色（而非完全委托给同步后端原生权限）。

## 1.2 v1 范围声明

| 本次设计包含                          | 明确不含（见 §12 Future Work）                  |
|---------------------------------|------------------------------------------|
| 邀请制工作空间 + 角色权限<br>GitHub 仓库同步后端 | 真正对外公开可发现的工作空间<br>实时协作光标 / 同时编辑同一字段的即时广播 |
| 云盘文件夹同步后端                       | 端到端加密（本设计预留密钥管理钩子，v1 默认不开启）              |
| 基于 oplog 的异步合并 + 冲突提示           | WebDAV / Syncthing 后端（架构可扩展，暂不实现）        |

与现有重构路线图的关系

本设计是《readme/refactor/redesign.tex》中 **P4（同步就绪）** 阶段预埋的版本号、墓碑表（tombstones）、操作日志（oplog）三项机制的**直接消费方**。这些机制在 P2 重构时就已经写入 db/migrations 与 Notifier 事件总线，本次不再重新设计数据模型，只是把”谁来读/写 oplog”这件事从”本地进程自己”扩展为”多台设备 + 多个协作者”。

# 2 总体架构：控制面与数据面分离

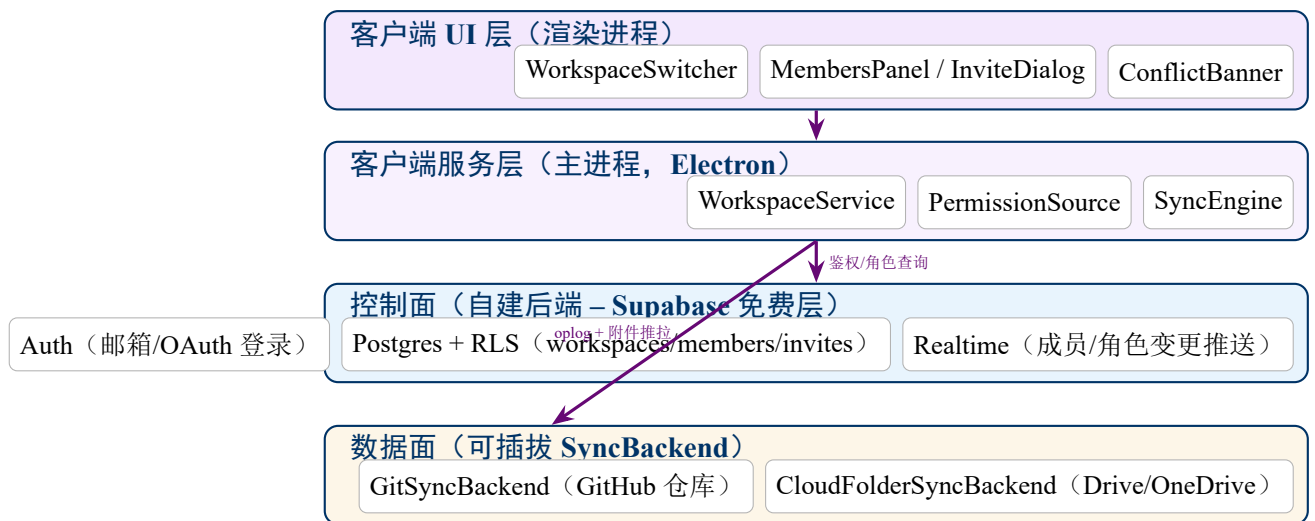
这是本设计的核心决策。把系统拆成两个职责完全独立、可分别替换的平面：

|        | 控制面 (Control Plane)   | 数据面 (Data Plane)          |
|--------|-----------------------|---------------------------|
| 职责     | 身份、工作空间成员、角色、邀请令牌     | 实际文献数据（oplog + 附件二进制）     |
| 存储位置   | 自建后端（推荐 Supabase 免费层） | GitHub 仓库 或 用户自己的云盘文件夹    |
| 数据量    | 极小（几十 KB 级的成员/角色表）    | 大（PDF、Markdown、图片，GB 级）   |
| 是否收费   | 免费层足够（见 §11 成本表）      | 完全免费——复用用户已有的 GitHub/云盘账号 |
| 单点故障风险 | 有（但数据量小，易备份/迁移）       | 无——数据本体始终在用户自己的仓库/网盘里     |

### 为什么要分离

控制面只回答”你是谁、你在哪些工作空间、你有什么权限”这类小数据、强一致性问题，适合放在一个真正的关系型数据库里做行级安全策略（RLS）。数据面只回答”字节怎么从 A 设备搬到 B 设备”这类大数据、最终一致性问题，适合交给用户已有的免费存储（GitHub/Google Drive/OneDrive）。两者用不同的一致性模型、不同的容量规划、不同的失败模式——混在一起会让 GitHub 仓库既要当”数据库”又要当”文件仓库”，两头都做不好。

## 2.1 分层架构图



## 3 控制面设计：工作空间、成员、角色

### 3.1 为什么选 Supabase

| 维度   | 说明                                                                                                            |
|------|---------------------------------------------------------------------------------------------------------------|
| 数据模型 | Postgres 关系型数据库，工作空间/成员/角色天然关系型数据，比 Firebase Firestore 的文档模型更贴合，也更接近 Veridian 本地 SQLite 的建模习惯，schema 可以直接照抄改写 |
| 权限实现 | 内置行级安全策略（ <b>Row Level Security</b> ），可以在数据库层再加一道”用户只能读自己所在工作空间的行”的防线，不完全依赖应用层代码                              |
| 免费层  | 500 MB 数据库 + 5 万月活用户 + 内置 Auth + Realtime 推送，对于”成员表 + 角色表”这种量级的数据完全够用，个人/小团队永远用不满                             |
| 账号体系 | 内置邮箱密码 / Magic Link / GitHub OAuth 登录，GitHub OAuth 登录还能顺便一次性拿到 GitHub 授权，给后面接 GitSyncBackend 省一步              |

### 3.2 控制面数据模型

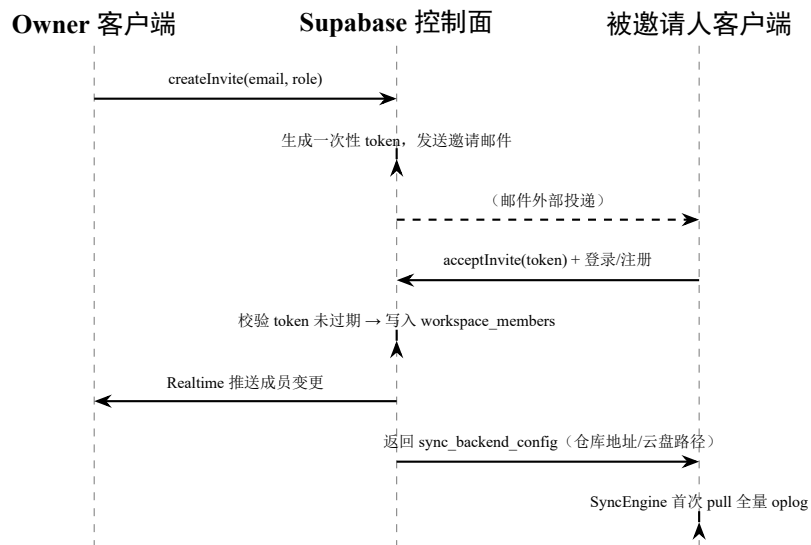
| 表                 | 字段说明                                                                                                                                              |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| workspaces        | id, name, kind ('private'   'shared'), owner_id, sync_backend_type, sync_backend_config (JSON, 仓库地址/云盘路径等非敏感连接信息; 令牌类敏感信息不落库, 见 §5.3), created_at |
| workspace_members | workspace_id, user_id, role (owner admin editor viewer), joined_at                                                                                |
| invites           | id, workspace_id, email, role, token (一次性), expires_at, status (pending accepted revoked)                                                         |
| devices           | id, user_id, device_name, last_synced_at (用于”私人工作空间”的多设备同步场景, 展示”我的设备列表”)                                                                         |

3.3 角色权限矩阵

| 操作               | Owner | Admin | Editor | Viewer |
|------------------|-------|-------|--------|--------|
| 查看文献/附件          | ✓     | ✓     | ✓      | ✓      |
| 新增/编辑/删除文献       | ✓     | ✓     | ✓      | ×      |
| 管理集合 (文件夹)       | ✓     | ✓     | ✓      | ×      |
| 邀请 Viewer/Editor | ✓     | ✓     | ×      | ×      |
| 邀请 Admin, 移除成员   | ✓     | ✓     | ×      | ×      |
| 修改同步后端配置         | ✓     | ×     | ×      | ×      |
| 删除工作空间           | ✓     | ×     | ×      | ×      |

角色定义是一张静态的角色 → 权限集合映射表 (见 §7 ISP/OCP 小节), 新增角色 (如未来的”Commenter”) 只需扩表, 不需要改任何判断权限的调用点。

### 3.4 邀请流程时序



被邀请人客户端拿到的是**连接信息**（仓库地址、云盘路径），而不是数据本体——数据本体在首次 pull 时才从数据面拉取，控制面全程不经手任何文献数据。

## 4 数据面设计：可插拔同步后端

### 4.1 SyncBackend 抽象

```

1 // shared/sync/SyncBackend.ts -- 纯传输接口，不含任何权限概念
2 // （权限已经在 S3 的控制面解决，后端只负责字节搬运）
3 export interface SyncBackend {
4   testConnection(): Promise<boolean>
5
6   // 拉取自 cursor (GitSyncBackend 用 commit SHA,
7   // CloudFolderSyncBackend 用 manifest 版本号) 之后的新变更包
8   pull(cursor: string | null): Promise<{ envelopes: SyncEnvelope[];
9     nextCursor: string }>
10
11   // 推送本地产生的变更包（内含 oplog 条目 + 引用的附件内容哈希）
12   push(envelope: SyncEnvelope): Promise<{ ok: boolean; conflict?: boolean }>
13
14   // 按内容哈希上传/下载附件二进制（去重：哈希已存在则跳过上传）
15   uploadBlob(hash: string, data: Uint8Array): Promise<void>
16   downloadBlob(hash: string): Promise<Uint8Array>
17 }
18
19 export interface SyncEnvelope {
20   deviceId: string
21   workspaceId: string
22   seqRange: [number, number] // 对应本地 oplog.seq 区间
23   entries: OplogEntry[]      // 复用现有 oplog 表结构
24   ts: number
25 }
  
```

## 4.2 GitSyncBackend (GitHub 仓库)

- 使用 **isomorphic-git** (纯 JS 实现) 而非依赖系统 **git** 可执行文件——Electron 应用免于要求用户预装 **Git**, 跨平台打包更简单。
- 仓库布局 (内容寻址, 天然无 **git** 合并冲突):

```
<repo>/
├─ oplog/<device-id>/<seq-start>-<seq-end>.jsonl    # 追加写入
├─ blobs/<sha256前2位>/<sha256>                  # 附件二进制, 按哈希去重
└─ manifest.json                                   # 各设备已知的最新 seq (仅供快速跳过)
```

- PUSH** = 写入新的 **oplog** 分片 + 未见过哈希的 **blob** 文件 → **commit** → **push**。
- PULL** = **fetch** + **fast-forward merge**; 由于文件按设备目录、内容哈希隔离, 结构上不会产生 **git** 合并冲突 (这是“CRDT via git”的常见模式)。
- 认证: **GitHub Personal Access Token** (细粒度 **PAT**, 仅 **contents:write** 权限), 通过 **Electron safeStorage** 加密存储——复用现有 **MinerU API Token** 的加密落地模式 (见 **readme/refactor/redesign.tex §3**)。
- 大文件: 单文件 > 50 MB 时提示启用 **Git LFS** (**GitHub** 免费层: 1 GB 存储 + 1 GB/月带宽), 避免仓库体积膨胀影响 **clone** 速度。

## 4.3 CloudFolderSyncBackend (云盘文件夹)

- 核心思路**: **Veridian** 不直接对接任何云盘 API, 只是把上面同样的 **oplog/ blobs/ manifest.json** 布局, 写在用户指定的一个本地目录里——这个目录恰好是 **Google Drive / OneDrive / Dropbox** 桌面客户端正在同步的目录。真正的字节搬运完全交给用户已经装好、免费的云盘客户端, **Veridian** 只管本地读写。
- 用 **chokidar** 监听该目录, 检测到其他设备写入的新 **oplog** 分片后触发 **pull-replay**。
- 零 API Key、零 OAuth、零第三方服务账号**——这是“免费、不承担费用、方便操作”这条需求里契合度最高的方案, 因为几乎所有用户机器上都已经装了至少一种云盘客户端。
- 已知限制: **SQLite** 数据库文件本身不可以直接丢进云盘同步目录 (云盘客户端可能在数据库写入过程中读到半写状态导致损坏)——这正是为什么要通过 **oplog** 分片文件 (一次性追加写完才算存在) 而不是同步 **.db** 文件本身。

对比: 为什么两个后端接口能长得一模一样

**GitSyncBackend** 和 **CloudFolderSyncBackend** 底层设施完全不同 (**Git** 协议 vs 文件系统 + 第三方客户端), 但对上层 **SyncEngine** 暴露的接口完全一致——这正是 §7 里里氏替换原则 (**LSP**) 的直接体现: 换后端不需要改一行 **SyncEngine** 代码。

## 4.4 冲突解决策略 (v1: 最后写入优先 + 手动提示)

- Pull** 到的 **oplog** 条目通过与本地写入完全相同的 **Service** 层函数回放 (**ItemService.updateItem** 等), 复用 **P2** 重构里“单一写入口必发事件”的设计——远程变更到达后 **UI** 自动刷新, 无需为同步单独写一套 **UI** 逻辑。

- 2. 字段级最后写入优先：比较 `items.version` + 操作时间戳，新版本覆盖旧版本（与 Zotero 自身的合并策略一致）。
- 3. 真正的并发冲突（两台设备在离线期间都改了同一文献的同一字段）会被识别（版本号分叉）并放入 `ConflictBanner`，用户手动选择保留哪一份，而不是静默丢弃——这是 v1 里唯一需要人工介入的场景。

## 5 权限与安全

### 5.1 PermissionSource 抽象

```
1 // main/services/PermissionSource.ts
2 export interface PermissionSource {
3   getRole(workspaceId: string, userId: string): Promise<Role | null>
4   subscribeRoleChanges(workspaceId: string, cb: (change: RoleChange) => void)
5     : () => void
6 }
7 // 目前唯一实现：SupabasePermissionSource（查 workspace_members 表 +
8 // 订阅 Supabase Realtime）。未来若要支持"自托管控制面"，
9 // 只需新增一个实现，WorkspaceService 不用改。
```

### 5.2 令牌与密钥的存储原则

| 凭据                | 存储位置                  | 加密方式                   |
|-------------------|-----------------------|------------------------|
| Supabase 会话 token | 本地（Electron userData） | safeStorage 加密         |
| GitHub PAT        | 本地（每设备各自持有）           | safeStorage 加密，不同步到控制面 |
| 云盘文件夹路径           | 本地设置                  | 明文（非敏感）                |

端到端加密（Future Work，非 v1 必须）

v1 的 `oplog/blob` 内容在数据面（GitHub/云盘）中以明文存储，安全边界依赖”用户信任自己选择的仓库/网盘”。若未来需要更强隐私保证，可在 `workspaces` 表加一列 `encrypted_workspace_key`（用每个成员的公钥做信封加密），`SyncEnvelope` 序列化前先用该对称密钥加密——控制面和数据面提供方都只能看到密文。本设计已把该密钥字段的位置预留好，但不在 v1 实现范围内。

## 6 客户端架构改动清单

| 新增模块                                        | 职责                                                                        |
|---------------------------------------------|---------------------------------------------------------------------------|
| <code>main/services/WorkspaceService</code> | 工作空间 CRUD、成员邀请/移除、当前工作空间切换；唯一的写入口，调用后发 <code>workspace.changed</code> 域事件 |
| <code>main/services/PermissionSource</code> | 角色查询抽象 + Supabase 实现 (§5.1)                                               |



|                                              |                                                 |
|----------------------------------------------|-------------------------------------------------|
| main/sync/Engine.ts                          | 编排 pull/push 时机、调用 SyncBackend、驱动 oplog 回放、检测冲突 |
| main/sync/backends/GitSyncBackend.ts         | §4.5 发明                                         |
| main/sync/backends/CloudFolderSyncBackend.ts | §4.6 发明                                         |
| main/sync/OplogReplayer.ts                   | 把远程 oplog 条目映射回 Service 层调用                     |
| shared/sync/SyncBackend.ts                   | 传输接口定义 (§4.1)，主/渲染进程共享类型                        |
| renderer/.../WorkspaceSwitcher.tsx           | 页面上具样工作空间切换下拉                                   |
| renderer/.../InviteDialog.tsx                | 输入邮箱 + 选择角色 → 调用 WorkspaceService               |
| renderer/.../MembersPanel.tsx                | 成员列表 + 角色调整 + 移除（按权限矩阵置灰）                       |
| renderer/.../ConflictBanner.tsx              | 显示待手动解决的字级冲突                                    |

新增 IPC 通道遵循现有 shared/ipc-contract.ts 契约化模式（zod schema + 网关校验），不额外发明新的 IPC 约定。

## 7 七大设计原则逐一对应

| 原则           | 在本设计中的体现                                                                                                                                                          |
|--------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 单一职责原则 (SRP) | SyncEngine（何时同步、冲突判定）与 SyncBackend（字节怎么搬）职责完全分离；WorkspaceService（成员/角色管理）与 PermissionSource（角色查询）分离；OplogReplayer（把远程条目映射回本地调用）单独成一个模块，不与传输逻辑混在一起。每个模块只有一个改变它的理由。 |
| 开闭原则 (OCP)   | 新增同步后端（WebDAV、Syncthing）只需新写一个实现 SyncBackend 接口的类，SyncEngine/UI 不需要改一行；新增角色只需在角色 → 权限集合的静态映射表里加一行 (§3.3)，所有权限判断代码天然生效，不需要改判断逻辑本身。                                 |
| 里氏替换原则 (LSP) | GitSyncBackend 与 CloudFolderSyncBackend 对 SyncEngine 而言完全可互换 (§4.3 fixbox)：两者都保证 push 要么完整成功要么完整失败（不会半推送），pull 不会丢失条目——这两条前置/后置条件对所有实现都一致成立。                      |
| 接口隔离原则 (ISP) | SyncBackend 只包含传输方法（pull/push/blob 存取），不包含任何权限判断方法——权限完全隔离进独立的 PermissionSource 接口，两个接口的消费者（SyncEngine vs WorkspaceService）互不依赖对方用不到的方法。                          |

|                      |                                                                                                                                                                         |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 依赖倒置原则 (DIP)         | SyncEngine 和 WorkspaceService 依赖的是 SyncBackend/PermissionSource 这两个抽象接口，具体用哪个实现（Git/CloudFolder，Supabase/未来的自托管方案）由工作空间配置在运行时注入，高层模块不 import 任何具体后端类。                   |
| 迪米特法则 (LoD，最少知识原则)   | UI 组件只通过 WorkspaceService 的 IPC 契约通信，从不直接触碰 SyncBackend 内部（GitHub API 细节、云盘路径拼接）或 oplog 表结构——延续 P2 重构”单一写入口、薄 IPC 处理器”的既有纪律。                                            |
| 合成复用原则 (CARP，组合优于继承) | SyncEngine 是通过组合一个可替换的 SyncBackend + OplogReplayer + 冲突判定逻辑构成的，而不是为每种后端派生一个 GitSyncEngine extends BaseSyncEngine 子类——避免脆弱基类问题，每个组件可独立单元测试（呼应 redesign.tex §9 的测试金字塔目标）。 |

## 8 与现有重构路线图的衔接

| 阶段 | redesign.tex 中的产物            | 本设计如何复用                                                         |
|----|------------------------------|-----------------------------------------------------------------|
| P2 | Notifier 事件总线 + Service 层    | OplogReplayer 复用同一批 Service 函数回放远程变更，远程变更自动触发 UI 刷新，零新增前端状态管理代码 |
| P3 | DB Worker 线程 + UnitOfWork 事务 | SyncEngine 的 pull 回放批量操作复用同一套事务封装，避免半回放状态                       |
| P4 | 版本号 + 墓碑表 + oplog            | 本设计的数据面协议直接就是”如何搬运这张 oplog 表”，零数据模型变更                           |
| P5 | 托管存储布局 storage/<item-key>/   | 附件内容寻址哈希直接对该目录下的文件计算，天然对齐                                       |

## 9 分阶段实施路线图

| 阶段 | 内容                                                                   | 工作量   | 风险 |
|----|----------------------------------------------------------------------|-------|----|
| W1 | Supabase 项目搭建: schema + RLS + Auth 接入; 客户端登录/注册 UI                   | 3–4 天 | 低  |
| W2 | WorkspaceService + PermissionSource + 工作空间切换 UI (暂不接同步, 先验证控制面闭环)    | 4–5 天 | 低  |
| W3 | SyncBackend 抽象 + CloudFolderSyncBackend (先做这个, 零外部依赖, 验证 oplog 传输协议) | 4–5 天 | 中  |
| W4 | GitSyncBackend (isomorphic-git 接入 + PAT 加密存储 + LFS 提示)               | 5–6 天 | 中  |
| W5 | 冲突检测 + ConflictBanner UI + 邀请流程端到端联调                                 | 4–5 天 | 中  |
| 测试 | 每阶段同步编写: SyncBackend 用假后端跑单测, OplogReplayer 用内存库跑集成测试                | 贯穿    | –  |

约 4 周完成 v1。W1–W2 (控制面) 与 W3–W4 (数据面) 在依赖上是并行的, 如有两位开发者可并行推进以压缩到约 2.5 周。

## 10 成本与限制

| 服务                      | 免费额度                                           | 对个人/小团队库的估算                                             |
|-------------------------|------------------------------------------------|---------------------------------------------------------|
| Supabase                | 500 MB 数据库, 5 万 MAU, 2 GB 文件存储 (本设计不用它存文献数据)   | 一个工作空间的成员/角色/邀请记录通常 < 1 MB; 千人协作规模才会接近上限                |
| GitHub (私有仓库)           | 免费账号: 无限私有仓库、无限协作者; LFS 免费层 1 GB 存储 + 1 GB/月带宽 | 纯文本 (oplog + markdown) 几乎不占额度; PDF 附件量大时需要开 LFS 或改用云盘后端 |
| Google Drive / OneDrive | 15 GB / 5 GB 免费额度                              | 覆盖大多数个人文献库; 团队场景建议每个协作者用自己的网盘额度存自己上传的原件                 |

### 何时需要考虑付费

单个工作空间协作者超过几百人, 或单个云盘账号存储的文献库超过其免费额度时, 才需要考虑: GitHub LFS 付费带宽包、云盘更高存储层级, 或自建 S3 兼容存储作为数据面第三种后端 (架构上只需新增一个 SyncBackend 实现)。对个人和小型科研团队场景, v1 设计**完全免费**。

## 11 待扩展功能（Future Work）

---

- 端到端加密（§5.2 已预留密钥字段）
- 真正对外公开可发现的工作空间（类似开源项目 + PR 审核合并模式）
- 实时协作（同一文献同时编辑时的即时光标广播，需要 WebSocket/CRDT，当前 oplog 异步合并模型不支持）
- WebDAV / Syncthing 作为第三、第四种 SyncBackend 实现
- 自托管控制面（PermissionSource 第二实现，替代 Supabase 依赖）
- 移动端 / Web 端客户端接入同一套控制面 + 数据面协议